claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-ada3a41a21fadb993.jsonl`
- SHA-256: `50e8e9f04b78a93ea068589333309e341eab8b8734789a8a42333d8118321cf6`
- Source modified: `2026-06-22T18:51:56+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T18:49:26.604Z)

```
Engine repo at C:/Users/avidu/Projects/bhi-computeval. Verify the COMPUTE PICKER + cloud path:
ProcessRequest.compute ("local"|"cloud") + deployment.cloud_available in /api/capabilities
(backend/main.py _cloud_available); how to CONFIGURE a server for cloud serving
(deployment.profile="cloud-serving" ? compute_target=cloud_run, storage.backend=gcs,
storage.output_root; env CLOUD_RUN_JOB/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT;
indexing.skip_ai_handoff); the Cloud Run Job deploy recipe (deploy/cloudrun/README.md +
Dockerfile); and scripts/verify_compute_path.py (what it asserts). Return the config + env
needed to light up the cloud toggle, and how a request picks cloud vs local. Report only
VERIFIED facts with file:line citations ? this doc must reflect SHIPPED state, never aspiration.
If something is NOT implemented, say so.
```

2. assistant (2026-06-22T18:49:29.888Z)

```
I'll verify the compute picker and cloud path in the engine repo. Let me start by pulling the
repo fresh and locating the relevant files.
```

3. user (2026-06-22T18:49:33.365Z)

```
Your configuration specifies to merge with the ref 'refs/heads/feat/compute-provenance'
from the remote, but no such ref was fetched.
---BRANCH---
feat/compute-provenance
ee6fba8 feat(serving): forward skip_ai_handoff to the cloud job so the picker's cloud path
yields rallies
4b6a560 feat(serving): compute-path provenance ? prove which backend ran the DETECT
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
```

4. user (2026-06-22T18:49:33.556Z)

```
total 304
drwxr-xr-x 1 avidu 197609     0 Jun 23 00:04 .
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 ..
-rw-r--r-- 1 avidu 197609 77824 Jun 23 00:02 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 22 23:41 .coveragerc
-rw-r--r-- 1 avidu 197609    90 Jun 22 23:41 .git
-rw-r--r-- 1 avidu 197609   947 Jun 22 23:41 .gitattributes
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 .github
-rw-r--r-- 1 avidu 197609  1304 Jun 22 23:41 .gitignore
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 .mypy_cache
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 .pytest_cache
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 .ruff_cache
-rw-r--r-- 1 avidu 197609  5525 Jun 22 23:41 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35329 Jun 22 23:41 README.md
```

```
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 __pycache__
drwxr-xr-x 1 avidu 197609     0 Jun 23 00:07 backend
-rw-r--r-- 1 avidu 197609   379 Jun 23 00:04 cloudbuild.cr.yaml
-rw-r--r-- 1 avidu 197609  2735 Jun 22 23:41 config.json
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 deploy
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 docs
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 22 23:41 mypy.ini
drwxr-xr-x 1 avidu 197609     0 Jun 23 00:09 output
-rw-r--r-- 1 avidu 197609  4525 Jun 22 23:41 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jun 22 23:41 pytest.ini
-rw-r--r-- 1 avidu 197609   642 Jun 22 23:41 requirements.txt
-rw-r--r-- 1 avidu 197609   884 Jun 22 23:41 ruff.toml
-rwxr-xr-x 1 avidu 197609  2055 Jun 22 23:41 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5620 Jun 22 23:41 run_eval.py
-rw-r--r-- 1 avidu 197609  4356 Jun 22 23:41 run_regression.py
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:41 scratch
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 scripts
drwxr-xr-x 1 avidu 197609     0 Jun 23 00:07 tests
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:50 tools
drwxr-xr-x 1 avidu 197609     0 Jun 22 23:51 training
```

5. assistant (2026-06-22T18:49:36.354Z)

The repo is on branch `feat/compute-provenance` (note the recent commits about compute-path provenance and the cloud picker). Let me read the key files.

6. user (2026-06-22T18:49:40.683Z)

```
---vs origin---
ee6fba8 feat(serving): forward skip_ai_handoff to the cloud job so the picker's cloud path
yields rallies
4b6a560 feat(serving): compute-path provenance ? prove which backend ran the DETECT
---status---
?? cloudbuild.cr.yaml
```

7. user (2026-06-22T18:49:40.960Z)

```
Dockerfile
README.md
---scripts---
verify_compute_path.py
---verify---
scripts/verify_compute_path.py
```

8. assistant (2026-06-22T18:49:44.645Z)

Two unmerged commits ahead of origin/master. Let me read the core files: `backend/main.py` (capabilities + cloud_available), the request model, the config schema, the deploy recipe, and the verify script.

9. user (2026-06-22T18:49:46.259Z)

```
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__init__.py
backend/config/loader.py
backend/config/models.py
backend/config/presets.py
backend/config/startup.py
backend/infrastructure/database.py
backend/main.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/worker/__main__.py
```

10. user (2026-06-22T18:49:49.211Z)

```
47:from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
270:    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
271:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
272:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
273:    compute: Optional[str] = None
348:def capabilities():
349:    """Honest probe of what THIS box can do ? the first-run wizard renders the compute step
from
376:            "compute_target": getattr(deployment, "compute_target", "") or "",
377:            # item 3: can this server satisfy a per-request compute="cloud" dispatch? The
SPA shows
379:            "cloud_available": _cloud_available(),
489:def _cloud_available() -> bool:
490:    """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item 3).
499:    if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
508:    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
515:    (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every non-``cloud_run``
target, so
517:    from backend.compute import ComputeJobSpec, get_compute_backend
529:                "compute": {"path": "cloud_run" if ran else "not_run", "requested":
req.compute,
532:    # SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
534:    # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
535:    choice = (req.compute or "").strip().lower()
537:        logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using
server default.",
538:                       req.compute)
543:        if not _cloud_available():
547:        compute = get_compute_backend(global_config, target_override="cloud_run")
549:        compute = get_compute_backend(global_config)  # server default (default-OFF unless
cloud_run)
550:    if compute is None:
581:        result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
608:        "requested": req.compute,
619:             "compute": provenance}, True)
642:    video_id = compute_video_id(local_video)
659:    compute_actual: Optional[str] = None
698:        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run
the heavy
706:            return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
708:        compute_actual = "in_process"
746:        # response["compute"] (path=cloud_run + the Cloud Run execution); for every other
path stamp
750:            response.setdefault("compute", {
751:                "path": compute_actual or "not_run",
752:                "requested": getattr(req, "compute", None),
754:        prov = response.get("compute", {}) if isinstance(response, dict) else {}
756:                    getattr(req, "compute", None), prov.get("path"), prov.get("dispatched"),
759:        # report.output_dir). Never raises. Surface the paths in the response. (The compute-
path
815:                                   locale=req.locale, compute=req.compute):
1122:    ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
1284:        video_id = compute_video_id(local_video)
```

11. assistant (2026-06-22T18:49:51.375Z)

Let me read the key sections of main.py in detail.

12. user (2026-06-22T18:49:52.448Z)

```
260
261     # --- API Models ---
262     class ProcessRequest(BaseModel):
263         video_path: str
264         player_pool: Optional[List[str]] = None
265         max_frames: Optional[int] = None
266         segmenter: Optional[str] = None
267         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
268         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
269         locale: Optional[str] = None
270         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
271         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
272         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
273         compute: Optional[str] = None
274
275     class QueryRequest(BaseModel):
276         video_id: str
277         server: Optional[str] = None
278         receiver: Optional[str] = None
279         duration_min: Optional[float] = None
280         event_type: Optional[str] = None
281
282     class CompileRequest(BaseModel):
283         video_id: str
284         segment_ids: List[int]
285         output_filename: str
286         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
287         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
288         locale: Optional[str] = None
289
290     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
291         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
292
293         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
294         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
295         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
296         """
297         if segments:
298             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
299         else:
300             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
301
302
303     # --- API Endpoints ---
304     @app.get("/api/config")
305     def get_config():
306         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
307         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
```

```
308         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
309         # by tests/test_redact.py + the /api/config redaction test).
310         if global_config is None:
311             return {}
312         return redact_secrets(global_config.model_dump(mode="json"))
313
314     @app.get("/api/health")
315     def health():
316         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
317         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
318         confirm the engine is reachable and which detector is wired. Never raises."""
319         sport = getattr(global_config, "sport", None)
320         indexing = getattr(global_config, "indexing", None)
321         return {
322             "status": "ok",
323             "sport": sport,
324             "default_segmenter": getattr(indexing, "default_segmenter", None),
325         }
326
327
328     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
329     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
330     API_VERSION = 1
331
332
333     def _engine_version() -> str:
334         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
335         try:
336             return _pkg_version("badminton-highlight-indexer")
337         except PackageNotFoundError:
338             return "unknown"
339
340
341     @app.get("/api/version")
342     def api_version():
343         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
344         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
_bundled_ui_version()}
345
346
347     @app.get("/api/capabilities")
348     def capabilities():
349         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
350         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
351         value. Best-effort + never raises (the wizard degrades gracefully)."""
352         indexing = getattr(global_config, "indexing", None)
353         deployment = getattr(global_config, "deployment", None)
354         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
355         ffprobe = shutil.which(ffprobe_bin())
356         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
357         wasb = getattr(indexing, "wasb", None)
358         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
359         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
360         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
361         # real GPU box) and importing it would add seconds to this probe. The detector
```

```
healthcheck is the
362         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
363         nvidia_smi = shutil.which("nvidia-smi")
364         return {
365             "api_version": API_VERSION,
366             "engine_version": _engine_version(),
367             "segmenters": sorted(segmenter_registry.list_available().keys()),
368             "default_segmenter": getattr(indexing, "default_segmenter", None),
369             "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
370                        "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
371             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
372             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
373             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
374             "deployment": {
375                 "profile": getattr(deployment, "profile", "") or "",
376                 "compute_target": getattr(deployment, "compute_target", "") or "",
377                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
378                 # the "run in the cloud" toggle only when true (else it's local-only).
379                 "cloud_available": _cloud_available(),
380             },
381         }
382
383
384     @app.get("/api/videos")
385     def list_videos(limit: Optional[int] = None):
386         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
387         single-row get existed (PACKAGING_PLAN.md P1)."""
388         return {"videos": db_instance.list_videos(limit=limit)}
389
390
391     def _reels_dir() -> str:
392         return getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
393
394
395     _REEL_EXTS = (".mp4", ".mov")
396
397
398     @app.get("/api/reels")
399     def list_reels():
400         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
401
402         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
403         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
404         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
405         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
406         base = _reels_dir()
407         reels = []
408         try:
409             entries = os.scandir(base)
410         except OSError:
411             return {"reels": []}
412         with entries:
413             for e in entries:
414                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
415                     try:
416                         st = e.stat()
```

```
417                     except OSError:
418                         continue
419                     reels.append({
```

13. assistant (2026-06-22T18:49:58.173Z)

Now let me read the `_cloud_available` function and the `_maybe_dispatch_remote` function in detail.

14. user (2026-06-22T18:49:59.405Z)

```
489     def _cloud_available() -> bool:
490         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
491
492         True when the control plane is wired for Cloud Run: either the configured default
target is
493         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
494         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
495         are present so a forced per-request override can actually dispatch + collect a
result.
496
497         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
498         deployment = getattr(global_config, "deployment", None)
499         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
500             return True
501         has_job = bool(os.environ.get("CLOUD_RUN_JOB") and
os.environ.get("GOOGLE_CLOUD_PROJECT"))
502         out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
503         return bool(has_job and out_root)
504
505
506     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
507                                play_wm: int, cand_wm: int, notify) ->
Optional[Tuple[Dict[str, Any], bool]]:
508         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
509         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
510         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
511
512         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
513         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
514         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
515         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
516         the in-process path stays byte-identical)."""
517         from backend.compute import ComputeJobSpec, get_compute_backend
518
519         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
520             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
521             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
522             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
523             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
524             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
```

```
rc / ingest
525             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
526             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
527             return ({"video_id": video_id, "status": "failed", "message": msg,
528                      "results": [r.model_dump() for r in val_results], "play_segments": [],
529                      "compute": {"path": "cloud_run" if ran else "not_run", "requested":
req.compute,
530                                  "target": "cloud_run", "dispatched": ran}}, ran)
531
532         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
533         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
534         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
535         choice = (req.compute or "").strip().lower()
536         if choice and choice not in ("local", "cloud"):
537             logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud');
using server default.",
538                            req.compute)
539             choice = ""
540         if choice == "local":
541             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
542         if choice == "cloud":
543             if not _cloud_available():
544                 return _failed("cloud-serving was requested but this server isn't configured
for it "
545                                "(no Cloud Run target). Pick 'run on my machine', or
configure cloud serving.",
546                                False)
547             compute = get_compute_backend(global_config, target_override="cloud_run")
548         else:
549             compute = get_compute_backend(global_config)  # server default (default-OFF
unless cloud_run)
550         if compute is None:
551             return None  # default-OFF ? run the in-process segmenter below (unchanged)
552
553         storage_cfg = getattr(global_config, "storage", None)
554
555         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
556         try:
557             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
558         except ValueError as e:
559             return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}",
False)
560         if input_ref.backend == "local":
561             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
562                            "can't be read by the Cloud Run job. Put the video in your bucket
first.", False)
563         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
564         if not out_root:
565             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
566                            "? that's where the Cloud Run job writes the result.", False)
567
568         notify("dispatching (cloud_run)")
569         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
570         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
```

```
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        sk = getattr(getattr(global_config, "indexing", None), "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name,
                              config_overrides=tuple(overrides))
        try:
            result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning("[API] cloud video_id %s != local %s ? input bytes differ between
the API hash "
                           "and the worker hash; ingesting under the local id.",
result.video_id, video_id)

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
        try:
            _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(f"cloud-serving: could not read the result from
{result.outputs_uri}: {e}", True)

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
        _finalize_reingest(video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info("[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
                    result.execution, provenance["job"], provenance["region"],
result.outputs_uri, video_id)
        return ({"video_id": video_id, "status": "success",
```

```
617                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run
/ '{seg_name}').",
618                 "results": [r.model_dump() for r in val_results], "play_segments":
segments,
```

15. assistant (2026-06-22T18:50:06.021Z)

Now let me read the `/api/process` flow where `_maybe_dispatch_remote` is called, and the compute backend factory.

16. user (2026-06-22T18:50:06.912Z)

```
618                 "results": [r.model_dump() for r in val_results], "play_segments":
segments,
619                 "compute": provenance}, True)
620
621
622     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
623         """The full hash ? validate ? segment ? report pipeline for one video.
624
625         This is the former synchronous /api/process body, unchanged in behavior, now run on
626         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
627         the sync endpoint used to return (UI compatibility); the per-video observability
628         report is still emitted in `finally:` and grafted as response["reports"].
629
630         `progress` is an optional callable(stage: str) used to surface coarse job progress.
631         Raises on unexpected internal errors ? the caller records those as a job failure
632         (after this function has already restored the pre-run segments, see below).
633         """
634         notify = progress or (lambda stage: None)
635
636         notify("hashing")
637         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
638         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
639         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
640         # consumers (hash / validators / segmenter) use the resolved local path.
641         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
642         video_id = compute_video_id(local_video)
643         was_reingest = db_instance.get_video(video_id) is not None
644
645         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
646         notify("validating")
647         logger.info(f"Running validations for {req.video_path}...")
648         val_results, val_context = registry.run_all_with_context(local_video, global_config)
649         failures = [r for r in val_results if not r.passed]
650
651         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
652         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
653         seg_name = req.segmenter or default_seg
654         segmenter_actual = None
655         segmentation_ran = False
656         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
657         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
658         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
659         compute_actual: Optional[str] = None
660         response: Optional[Dict[str, Any]] = None
```

```
661         try:
662             if failures:
663                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
664                 # status; it no longer destroys the video's prior good segments.
665                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
666                 response = {
667                     "video_id": video_id,
668                     "status": "failed",
669                     "message": "Video failed validation checks.",
670                     "results": [r.model_dump() for r in val_results],
671                 }
672                 return response
673
674             # 2. Run segmenter & indexer
675             logger.info("[API] Video passed checks. Segmenting and indexing...")
676             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
677
678             segmenter_cls = segmenter_registry.get(seg_name)
679             if not segmenter_cls:
680                 # Submit-time validation already 400s unknown names; this is the defensive
681                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
682                 available = list(segmenter_registry.list_available().keys())
683                 response = {
684                     "video_id": video_id,
685                     "status": "failed",
686                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
687                     "results": [r.model_dump() for r in val_results],
688                     "play_segments": [],
689                 }
690                 return response
691
692             logger.info(f"[API] Using segmenter: {seg_name}")
693             notify(f"segmenting ({seg_name})")
694             segmenter_actual = seg_name
695             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
696             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
697             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
698             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
699             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
700             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
701             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
702             # they're dropped on the cloud path in this slice ? a documented follow-up.)
703             remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
704             if remote is not None:
705                 response, segmentation_ran = remote  # `ran` = did the cloud job actually
execute (report)
706                 return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
707             # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
708             compute_actual = "in_process"
709             segmenter = segmenter_cls(db_instance, global_config)
710             try:
711                 result = segmenter.process_video(video_id, local_video, req.player_pool,
```

```
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault("compute", {
                    "path": compute_actual or "not_run",
                    "requested": getattr(req, "compute", None),
                })
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info("[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s
status=%s", video_id,
                        getattr(req, "compute", None), prov.get("path"),
prov.get("dispatched"),
                        (response or {}).get("status") if isinstance(response, dict) else
None)
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
```

```
764                 val_results=val_results, val_context=val_context,
765                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
766                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
767             )
768             if paths and isinstance(response, dict):
769                 response["reports"] = paths
770
771
772     @app.post("/api/process", status_code=202)
773     def process_video(req: ProcessRequest, request: Request):
774         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
775
776         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
777         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
```

17. assistant (2026-06-22T18:50:10.529Z)

Now the compute backend factory and the cloud_run backend.

18. user (2026-06-22T18:50:11.258Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38          target_override: Optional[str] = None,
39      ) -> Optional[ComputeBackend]:
40          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
```

```
to run
41          in-process (the default-OFF path).
42
43          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
44          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
45          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
46          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
47
48          ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
49          per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
50          server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
51          """
52          target = (
53              target_override
54              if target_override is not None
55              else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
56          )
57          if target != "cloud_run":
58              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
59
60          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
61          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
62
63          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
64          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
65          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
66          client = run_client if run_client is not None else GoogleCloudRunJobClient()
67          return CloudRunCompute(
68              run_client=client, job_name=job_name, region=region, project=project,
69              storage_config=_storage_dict(config), policy=policy, token=token,
70          )
71
```

19. assistant (2026-06-22T18:50:13.765Z)

Now the config models for the `deployment` and `storage` sections, plus the `cloud-serving` preset/startup.

20. user (2026-06-22T18:50:15.633Z)

```
36:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
57:    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
63:    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
130:    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
192:    # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
202:    skip_ai_handoff: bool = False
234:    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
240:    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
272:    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
307:    # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
311:class StorageConfig(BaseModel):
312:    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
315:    Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the
constructor ?
317:    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318:    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
```

```
320:    output_root: str = ""
321:    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
325:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
328:    # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329:    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
337:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
347:    # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive scope.
351:class DeploymentConfig(BaseModel):
352:    """Deployment profile ? one switch that selects a coherent bundle of compute + storage
355:    **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
356:    behaves exactly as before this section existed. A profile is opt-in (``config.json``
357:    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
359:    these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
360:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
362:    Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
364:    # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
365:    profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
366:    # Where the core runs. Set by the profile preset; overridable in config.json. Inert in
M1
368:    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
```

21. user (2026-06-22T18:50:18.722Z)

```
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
```

```
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
347         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
348         gdrive_api: dict[str, Any] = Field(default_factory=dict)
349
350
351     class DeploymentConfig(BaseModel):
352         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
353         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
354
355         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
356         behaves exactly as before this section existed. A profile is opt-in (``config.json``
357         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
358         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
359         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
360         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
361
362         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
363
364         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
365         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
366         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
367         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
368         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
369
370
371     class BillingConfig(BaseModel):
372         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
373
374         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
375         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
376         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
377         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
378         inserts a real, calibrated version and points ``pricebook_version`` at it.
379
380         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
381         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
382         pricebook + the recording seam; usage emission (the metering hooks) is a separate
```

```
step."""
383
384         enabled: bool = False
385         pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to
a real one
```

22. assistant (2026-06-22T18:50:20.962Z)

Now the presets file to see what `cloud-serving` profile sets.

23. user (2026-06-22T18:50:21.705Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
```

```
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61              "storage": {"backend": "local"},
62          },
63      }
64
65
66      def is_known_profile(profile: str) -> bool:
67          """True for a non-empty, recognised profile name."""
68          return bool(profile) and profile in PROFILE_PRESETS
69
70
71      def preset_for(profile: str) -> dict[str, Any]:
72          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73          module-level table), or ``{}`` for ``""`` / an unknown profile."""
74          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77      def _truthy(val: Optional[str]) -> bool:
78          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79          if val is None:
80              return False
81          return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84      def _cuda_available() -> bool:
85          """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87          try:
88              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89              # and optional). Only reached when a caller opts into probe_cuda.
90              import torch
91
92              return bool(torch.cuda.is_available())
93          except Exception:
94              return False
95
96
97      def probe_cuda_available() -> bool:
98          """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101         that *wants* the GPU heads-up can pass the result in without the startup module
```

```
importing
102         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105         error that could false-fail."""
106         return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:
112         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114         Signals, in priority order:
115
116           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
117           - ``CI`` truthy                        ? ``"cpu-mock"``
118           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119           - otherwise                            ? ``""`` (no strong suggestion)
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124         """
125         e = os.environ if env is None else env
126         if e.get("K_SERVICE"):
127             return "cloud-serving"
128         if _truthy(e.get("CI")):
129             return "cpu-mock"
130         if probe_cuda and _cuda_available():
131             return "truly-local"
132         return ""
133
```

24. assistant (2026-06-22T18:50:23.870Z)

Now the cloud_run backend implementation, the deploy README + Dockerfile, and the verify script.

25. user (2026-06-22T18:50:24.550Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
```

```
    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` + native detector)
    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just computes.

    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole shim is unit-tested
    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
    owner-verified on the M7 real run.
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, List, Optional

    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy, poll_until
    from backend.storage import fetch, parse_uri
    from backend.storage import get_storage

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test dependency)."""

        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
```

```
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77              )
78              op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79              return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82      class CloudRunCompute(ComputeBackend):
83          """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85          def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                       project: Optional[str] = None, storage_config: Optional[dict] = None,
87                       policy: ExecutionPolicy = DEFAULT_POLICY,
88                       token: Optional[str] = None,
89                       container_overrides: Optional[tuple[str, ...]] = None) -> None:
90              self._client = run_client
91              self._job_name = job_name
92              self._region = region
93              self._project = project
94              self._storage_config = storage_config or {}
95              self._policy = policy
96              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98              self._token = token
99              self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                                "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                              project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                      self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
```

```
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report,
133                                  execution=str(execution or ""))
134
135         # --- storage-backed completion signal (bucket-poll) ---
136         def _read_marker(self, done_uri: str) -> Optional[dict]:
137             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
138             ref = parse_uri(done_uri)
139             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
140                 return None
141             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
142
143         def _read_json(self, uri: str) -> dict:
144             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
145             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
146             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
147             try:
148                 ref = parse_uri(uri)
149                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
150                 local = fetch(uri, dst, config=self._storage_config)
151                 with open(local, "r", encoding="utf-8") as f:
152                     data: Any = json.load(f)
153                 return data if isinstance(data, dict) else {}
154             except Exception:
155                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
156                 return {}
157
```

26. assistant (2026-06-22T18:50:28.407Z)

Now the deploy README, the Dockerfile, and the verify script.

27. user (2026-06-22T18:50:29.907Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8       > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
```

```
confirm cmd + hourly
9       > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11      ## 0. Prerequisites (owner)
12      - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13      - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14        `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15      - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17      ## 1. Build + push the image (Artifact Registry)
18      ```bash
19      PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20      gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21      gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22      ```
23      *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25      ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26      ```bash
27      gcloud run jobs create rally-worker \
28        --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29        --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30        --max-retries 0 --task-timeout 3600 \
31        --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32      # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33      ```
34
35      ## 3. Wire the dispatcher (control plane)
36      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38      ```bash
39      export DEPLOYMENT_PROFILE=cloud-serving
40      export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44      ```
45
46      ## 4. M7 acceptance (the owner runlog)
47      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52      ## Gotchas
53      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
```

```
Run API status
56        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
58        real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
59        video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
60        (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
61        surfaces it as a `cloud dispatch error` job failure (not a traceback).
62      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
63
```

28. user (2026-06-22T18:50:32.668Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13      #
14      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20      ENV DEBIAN_FRONTEND=noninteractive \
21          PYTHONUNBUFFERED=1 \
22          PIP_NO_CACHE_DIR=1 \
23          CONDA_DIR=/opt/conda \
24          WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27      RUN apt-get update && apt-get install -y --no-install-recommends \
28              ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29              python3 python3-pip python3-venv \
30          && rm -rf /var/lib/apt/lists/*
31
32      # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
33      # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
34      # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
```

```
Anaconda-ToS-gated
35      # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
36      # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
37      # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
38      ENV MAMBA_ROOT_PREFIX=/opt/conda
39      RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
40              | tar -xj -C /usr/local bin/micromamba \
41          && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
42          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
43          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
44              torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
45          && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
46          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
47              hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
48
49      # --- Main application env ---
50      WORKDIR /app
51      COPY pyproject.toml README.md ./
52      COPY backend ./backend
53      COPY training ./training
54      # Install with the [storage-gcs] extra: the cloud worker PULLS the gs:// input and
PUSHES the
55      # gs:// outputs/done-marker through backend.storage (google-cloud-storage is lazy-
imported there).
56      # Without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
google.cloud` ? the
57      # real-Cloud-Run failure the mock-client tests can't see (the bucket I/O is faked in
tests).
58      RUN python3 -m pip install --upgrade pip && python3 -m pip install ".[storage-gcs]"
59
60      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
61      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
62      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
63          WASB_REPO_DIR=/opt/models/WASB-SBDT \
64          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
65          RALLY_DETECTOR_IMPL=native
66
67      # The dispatcher (CloudRunCompute) supplies the per-execution args:
68      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
69      ENTRYPOINT ["python3", "-m", "backend.worker"]
70
```

29. assistant (2026-06-22T18:50:34.540Z)

Now the verify script.

30. user (2026-06-22T18:50:35.173Z)

```
1       #!/usr/bin/env python3
2       """Verify the compute path an /api/process job ACTUALLY took (COMPUTE_DECOUPLED
validation method).
3
4       Submit a video with a chosen compute target, poll the job, and ASSERT
5       ``result["compute"]["path"]`` matches the intent ? then, for a cloud run, independently
cross-check
6       that the recorded Cloud Run **execution** really exists in GCP (`gcloud run jobs
```

```
executions describe`).
7       This closes the loop: the picker asked for a path, and we prove the engine took it (no
silent fallback).
8
9       Exit 0 = the intended path was provably taken; non-zero = a mismatch / the cloud
execution couldn't
10      be confirmed. Stdlib-only (urllib) so it runs anywhere; `--verify-gcp` shells out to
`gcloud`.
11
12      Examples:
13        # local engine, cloud path, with the GCP cross-check
14        python scripts/verify_compute_path.py --base http://127.0.0.1:8000 \
15            --video gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4 \
16            --segmenter wasb_hybrid --compute cloud --expect cloud_run --verify-gcp \
17            --region asia-southeast1 --project gen-lang-client-0306026826
18
19        # prove a local request stays in-process
20        python scripts/verify_compute_path.py --base http://127.0.0.1:8000 \
21            --video gs://b/clip.mp4 --compute local --expect in_process
22      """
23      import argparse
24      import json
25      import subprocess
26      import sys
27      import time
28      import urllib.request
29
30
31      def _post(base: str, path: str, body: dict):
32          req = urllib.request.Request(base.rstrip("/") + path,
data=json.dumps(body).encode(),
33                                       headers={"Content-Type": "application/json"},
method="POST")
34          with urllib.request.urlopen(req, timeout=30) as r:  # noqa: S310 ? operator-supplied
base URL
35              return r.status, json.loads(r.read().decode())
36
37
38      def _get(base: str, path: str):
39          with urllib.request.urlopen(base.rstrip("/") + path, timeout=30) as r:  # noqa: S310
40              return r.status, json.loads(r.read().decode())
41
42
43      def main() -> int:
44          ap = argparse.ArgumentParser(description="Prove which compute path an /api/process
job took.")
45          ap.add_argument("--base", required=True, help="engine base URL, e.g.
http://127.0.0.1:8000")
46          ap.add_argument("--video", required=True, help="video_path (a gs:// URI for the
cloud path)")
47          ap.add_argument("--segmenter", default=None)
48          ap.add_argument("--compute", choices=["local", "cloud"], default=None,
49                          help="the picker choice to send (omit ? server default)")
50          ap.add_argument("--expect", choices=["in_process", "cloud_run"], required=True,
51                          help="the path the job MUST report taking")
52          ap.add_argument("--region", default="asia-southeast1", help="fallback Cloud Run
region for --verify-gcp")
53          ap.add_argument("--project", default=None, help="GCP project for --verify-gcp")
54          ap.add_argument("--verify-gcp", action="store_true",
55                          help="cross-check the recorded Cloud Run execution exists in GCP
(gcloud)")
56          ap.add_argument("--timeout-sec", type=float, default=1200)
57          args = ap.parse_args()
58
59          body: dict = {"video_path": args.video}
```

```
60          if args.segmenter:
61              body["segmenter"] = args.segmenter
62          if args.compute:
63              body["compute"] = args.compute
64
65          status, sub = _post(args.base, "/api/process", body)
66          if status != 202:
67              print(f"FAIL submit: HTTP {status}: {sub}")
68              return 2
69          job_id = sub["job_id"]
70          print(f"submitted job {job_id}; polling /api/jobs/{job_id} ?")
71
72          started = time.monotonic()
73          st, job = "queued", {}
74          while st not in ("done", "failed"):
75              if time.monotonic() - started > args.timeout_sec:
76                  print("FAIL: job poll timed out")
77                  return 3
78              time.sleep(5)
79              _, job = _get(args.base, f"/api/jobs/{job_id}")
80              st = job.get("status", "queued")
81              print(f"  {st} ? {job.get('progress')}")
82
83          result = job.get("result") or {}
84          prov = result.get("compute") or {}
85          print(f"job {st}; result.status={result.get('status')}; compute={json.dumps(prov)}")
86
87          actual = prov.get("path")
88          if actual != args.expect:
89              print(f"FAIL: intended path {args.expect!r} but the job took {actual!r}")
90              return 4
91          print(f"OK: the job took the intended compute path: {actual}")
92
93          if args.expect == "cloud_run" and args.verify_gcp:
94              execu = prov.get("execution")
95              if not execu:
96                  print("FAIL: cloud_run path but no execution name recorded ? cannot cross-
check")
97                  return 5
98              cmd = ["gcloud", "run", "jobs", "executions", "describe", execu,
99                     "--region", prov.get("region") or args.region,
100                    "--format",
"value(metadata.name,status.succeededCount,status.failedCount)"]
101             if args.project:
102                 cmd += ["--project", args.project]
103             print("cross-checking in GCP:", " ".join(cmd))
104             cp = subprocess.run(cmd, capture_output=True, text=True)  # noqa: S603
105             if cp.returncode != 0:
106                 print(f"FAIL: GCP could not confirm execution {execu}: {cp.stderr.strip()}")
107                 return 6
108             print(f"OK: GCP confirms the Cloud Run execution {execu}: {cp.stdout.strip()}")
109
110         print("VERIFIED ?  the intended compute path was provably taken.")
111         return 0
112
113
114     if __name__ == "__main__":
115         sys.exit(main())
116
```

31. assistant (2026-06-22T18:50:38.398Z)

Now let me verify a few remaining details: the loader applying the preset, the worker `infer` CLI (cmd_infer dispatch guard + done-marker), and `resolve_input_ref`. Let me also confirm the `cloudbuild.cr.yaml` (untracked) and check the worker.

32. user (2026-06-22T18:50:40.958Z)

```
24:    PROFILE_PRESETS,
26:    preset_for,
39:def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
47:            out[key] = _deep_merge(out[key], val)
58:    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
62:    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
68:            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
69:            "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
108:def load_config(path: str | Path | None = None) -> AppConfig:
151:    # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With
no
162:            profile, ", ".join(PROFILE_PRESETS),
167:        merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
173:            ct = merged["deployment"].get("compute_target")
176:                # An active profile with no explicit compute_target (e.g. config.json
blanked it
178:                # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179:                merged["deployment"]["compute_target"] = canonical
182:                    "[CONFIG] deployment profile %r normally uses compute_target=%r but "
186:            logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187:                        profile, merged["deployment"].get("compute_target"))
195:                "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
217:# local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
```

33. user (2026-06-22T18:50:43.058Z)

```
1:"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
4:        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
48:    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
61:    """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
65:            raise ValueError(f"--set expects k=v, got {kv!r}")
118:def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
120:    outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121:    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.
124:    poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
138:                result.video_id, result.returncode, result.outputs_uri)
139:    return result.returncode
142:def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count: int,
144:    """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
145:    dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
147:        marker = {"video_id": video_id, "returncode": returncode,
152:        storage.put(local, done_uri, config=storage_cfg)
153:        logger.info("[worker] wrote completion marker -> %s", done_uri)
155:        logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
158:def cmd_infer(args: argparse.Namespace) -> int:
164:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
165:    # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through
to the
166:    # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
167:    # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
170:        return _dispatch_remote(compute, args)
194:        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until its
```

```
poll
195:        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
196:        if getattr(args, "done_uri", None):
197:            _write_done_marker(args.done_uri, video_id, rc, 0, "failed", storage_cfg,
scratch)
263:    # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
264:    # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker FAILURES
(rc 2/3)
265:    # write their own marker via _fail() above, so the dispatcher fails FAST either way (no
30-min hang
266:    # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local
run).
267:    if getattr(args, "done_uri", None):
268:        _write_done_marker(args.done_uri, video_id, 0, len(segments), status, storage_cfg,
scratch)
463:    rc = subprocess.run(cmd).returncode
496:    pi.add_argument("--done-uri", default=None,
498:                            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll
terminates. "
500:    pi.add_argument("--set", action="append", default=[], metavar="k=v",
501:                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
```

34. assistant (2026-06-22T18:50:47.267Z)

Let me verify the cmd_infer dispatch guard at lines 158-172 and the cloudbuild file, plus check resolve_input_ref exists.

35. user (2026-06-22T18:50:48.227Z)

```
158     def cmd_infer(args: argparse.Namespace) -> int:
159         config = load_config(args.config) if args.config else load_config()
160         _apply_overrides(config, args.set)
161         if not _run_startup_checks(config):
162             return 2
163
164         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
165         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
166         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
167         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
168         compute = get_compute_backend(config)
169         if compute is not None:
170             return _dispatch_remote(compute, args)
171
```

36. user (2026-06-22T18:50:50.448Z)

```
1       steps:
2         - name: 'gcr.io/cloud-builders/docker'
3           args:
4             - build
5             - -t
6             - asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker:latest
7             - -f
8             - deploy/cloudrun/Dockerfile
9             - .
10      images:
11        - asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest
```

```
12      options:
13        machineType: E2_HIGHCPU_8
14      timeout: 3600s
15
```

37. assistant (2026-06-22T18:50:53.557Z)

Now let me confirm `resolve_input_ref` and `acquire_local_input` exist in storage, and check the test that exercises the cloud picker path, plus whether there's any owner-run deploy report indicating the real M7 run happened.

38. user (2026-06-22T18:50:56.876Z)

```
38:def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
59:def acquire_local_input(raw: str, storage_config: Any = None, *,
---runlogs---
README.md
RUNLOG_truly-local_TEMPLATE.md
---tests on picker---
tests/test_api_cloud_dispatch.py
tests/test_async_jobs.py
tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc
```

39. assistant (2026-06-22T18:50:59.727Z)

So no real M7 deploy runlog exists yet (only the template). Let me check the test to confirm what's actually exercised, and confirm the submit-time fast-fail path for `compute` in `/api/process`.

40. user (2026-06-22T18:51:01.377Z)

```
6:done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
18:import backend.compute as compute_pkg
20:from backend.compute.cloud_run import CloudRunJobClient
72:    """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
78:        "deployment": {"compute_target": "cloud_run"},
93:    """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote)
build a
95:    forwards the per-request ``target_override`` (item 3) so the cloud-picker path resolves
too."""
96:    real = compute_pkg.get_compute_backend
97:    monkeypatch.setattr(compute_pkg, "get_compute_backend",
106:def test_cloud_dispatch_ingests_segments(cloud_env):
118:    assert all(md["source"] == "cloud_run" for md in metas)
123:def test_cloud_dispatch_builds_correct_spec(cloud_env):
136:    assert "deployment.compute_target=local_gpu" in joined  # container runs INLINE (never
re-dispatch)
141:def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
152:def test_cloud_failure_marks_failed_no_rows(cloud_env):
164:    # It DID dispatch + run remotely (then failed rc=1), so the path is honestly cloud_run +
dispatched.
165:    assert out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
168:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
180:def test_default_off_runs_in_process(tmp_path, monkeypatch):
181:    """compute_target='' ? get_compute_backend returns None ? the in-process segmenter runs
184:    cfg = AppConfig()  # compute_target defaults to ""
203:    assert "cloud_run" not in out["message"]
206:def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
233:def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
255:# --- item 3: SPA compute-picker (per-request compute override + cloud_available capability)
---
257:def test_per_request_local_forces_in_process(cloud_env):
```

```
258:    """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
266:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion", compute="local"))
269:    assert "cloud_run" not in out["message"]
273:def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
274:    """Server default is LOCAL (compute_target=''), but a per-request compute='cloud' forces
a Cloud
280:        "deployment": {"compute_target": ""},  # server default = in-process
296:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="cloud"))
302:def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
303:    """compute='cloud' on a server NOT wired for it fails clearly (no silent local fallback,
no
306:    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}})  # no bucket, no
env
316:        m.ProcessRequest(video_path=str(video), segmenter="wasb_hybrid", compute="cloud"))
321:def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
322:    """An unrecognised compute value is ignored (logged) and the server default applies ?
here a
323:    cloud_run server still dispatches to the cloud."""
329:            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="banana"))
331:    assert len(fake.calls) == 1  # fell back to the configured cloud_run default
332:    assert any("unknown compute" in r.getMessage() for r in caplog.records)
335:def test_capabilities_reports_cloud_available(cloud_env):
336:    """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA
shows the
340:    assert caps["deployment"]["cloud_available"] is True
341:    assert caps["deployment"]["compute_target"] == "cloud_run"
344:def test_capabilities_cloud_unavailable_by_default(monkeypatch):
345:    """A default (local) server with no Cloud Run env advertises cloud_available=False."""
350:    assert caps["deployment"]["cloud_available"] is False
```

41. assistant (2026-06-22T18:51:07.088Z)

The picker path is well-tested. Let me run the cloud dispatch test suite to confirm green on this branch, and confirm the submit-time validation of `compute` in the `/api/process` endpoint body.

42. user (2026-06-22T18:51:14.643Z)

```
.......................                                                  [100%]
23 passed in 4.28s
```

43. assistant (2026-06-22T18:51:56.270Z)

All 23 tests pass. I have verified every component. Let me give the final report.

Compute Picker + Cloud Path ? VERIFIED state (engine repo `bhi-computeval`)

Branch caveat (read first): The cloud-picker provenance work is on branch `feat/compute-provenance`, **2 commits ahead of `origin/master`, NOT yet merged** (`4b6a560` compute-path provenance, `ee6fba8` forward `skip_ai_handoff`). The picker/dispatch core (`_maybe_dispatch_remote`, `_cloud_available`, `get_compute_backend`) and the `cloud-serving` profile predate this branch, but the `compute` field on `ProcessRequest`, the `cloud_available` capability, the `execution` provenance, and `scripts/verify_compute_path.py` live on this unmerged branch. The 23-test `tests/test_api_cloud_dispatch.py` suite passes locally. Tree also has an untracked `cloudbuild.cr.yaml`.

Reality check on "shipped": the dispatch shim + image + tests are landed and **mock-tested only**. The real Cloud Run L4 build/run (M7) is an **owner step that has NOT happened** ? `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` contains only `RUNLOG_truly-local_TEMPLATE.md` and the

README, no `DEPLOY_REPORT_cloud-serving_*`. The real GCP client
(`GoogleCloudRunJobClient.run_job`) is `# pragma: no cover` ? never exercised in CI.

How a request picks cloud vs local

`ProcessRequest.compute: Optional[str]` (`backend/main.py:273`), values `"local" | "cloud" |
None`. Resolved in `_maybe_dispatch_remote` (`backend/main.py:506`), called from
`_execute_processing` at `backend/main.py:703`:
- `"local"` ? `return None` ? forces in-process even on a cloud server (`main.py:540-541`).
- `"cloud"` ? guarded by `_cloud_available()`; if false, returns a clear `_failed(...)` with
`path="not_run"` (no silent local fallback) (`main.py:542-546`); if true,
`get_compute_backend(global_config, target_override="cloud_run")` (`main.py:547`).
- `None` ? server default `get_compute_backend(global_config)` (`main.py:549`), default-OFF
(None unless `compute_target=="cloud_run"`).
- unknown value (e.g. `"banana"`) ? warns and falls back to server default (`main.py:536-539`).

Provenance is stamped on every response (`response["compute"]`): cloud success carries
`path="cloud_run"`, `job`, `region`, `execution`, `outputs_uri` (`main.py:606-613`); in-process
sets `path="in_process"` (`main.py:708`); the `finally` block defaults `path` to
`not_run`/`in_process` + `requested` (`main.py:749-757`).

`cloud_available` in `/api/capabilities`

`backend/main.py:347` `capabilities()` returns `deployment.cloud_available = _cloud_available()`
(`main.py:379`). `_cloud_available()` (`main.py:489`) is True when **either**
`deployment.compute_target == "cloud_run"` (`main.py:499`) **or** (`CLOUD_RUN_JOB` AND
`GOOGLE_CLOUD_PROJECT` env set) AND `storage.output_root` non-empty (`main.py:501-503`). The SPA
shows the cloud toggle only when this is True.

How to CONFIGURE a server for cloud serving

Profile (`config.json` `deployment.profile="cloud-serving"` or env
`DEPLOYMENT_PROFILE=cloud-serving`). The preset (`backend/config/presets.py:51-56`) sets:
- `deployment.compute_target="cloud_run"`
- `indexing.detector_impl="native"`, `indexing.skip_ai_handoff=False`
- `storage.backend="gcs"`

Applied by the loader via `_deep_merge(defaults, preset, file_data)`
(`backend/config/loader.py:167`), with `compute_target` back-filled from the profile if blanked
(`loader.py:173-179`). Field defs: `DeploymentConfig.profile`/`compute_target` Literals
(`backend/config/models.py:365,368`); `StorageConfig.backend`/`output_root`
(`models.py:318-320`).

You must also set `storage.output_root` to a bucket (e.g. `gs://your-bucket`) ? the preset
does NOT set it; the dispatch hard-requires it (`main.py:563-566`) and the input must be a cloud
URI (`gs://?`), not a local path (`main.py:560-562`).

Env (control plane) consumed by `get_compute_backend` (`backend/compute/__init__.py:63-65`):
- `CLOUD_RUN_JOB` (default `"rally-worker"`)
- `CLOUD_RUN_REGION` (default `"asia-south1"`)
- `GOOGLE_CLOUD_PROJECT` (no default; the GCP project)

Note the README example uses region `asia-south1`/`CLOUD_RUN_JOB=rally-worker`, while
`verify_compute_path.py` defaults and `cloudbuild.cr.yaml` use `asia-southeast1` + project `gen-
lang-client-0306026826` ? region is owner-environment-specific, not a fixed value.

`indexing.skip_ai_handoff` is forwarded to the cloud job so it yields rallies directly from WASB
windows without a Gemini key (`main.py:574-577`); the container is forced INLINE via
`deployment.compute_target=local_gpu` + `indexing.detector_impl=native`
(`backend/compute/cloud_run.py:41-44`) so it never re-dispatches (guard at
`backend/worker/__main__.py:168-170`).

Minimal cloud-serving config:
```
DEPLOYMENT_PROFILE=cloud-serving
CLOUD_RUN_JOB=rally-worker
CLOUD_RUN_REGION=<your-region>
```

```
GOOGLE_CLOUD_PROJECT=<your-project>
# config.json: storage.output_root="gs://<bucket>"  (input must be a gs:// URI)
```

Cloud Run Job deploy recipe (`deploy/cloudrun/`)

`README.md` 4 steps: (1) `gcloud builds submit --tag ?/rally-worker:latest -f deploy/cloudrun/Dockerfile .`; (2) `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600`, staging `WASB_WEIGHTS`; (3) export the env above + dispatch via `worker infer` or the API; (4) capture an M7 deploy runlog. The `Dockerfile` (`deploy/cloudrun/Dockerfile`) is `nvidia/cuda:12.4.1-runtime-ubuntu22.04` with a micromamba `wasb` conda env (torch 1.11.0+cu113, ADR-005 BSD channels), the app installed with `.[storage-gcs]`, `ENTRYPOINT ["python3","-m","backend.worker"]`. **WASB-C3 caveat (Dockerfile:14-16):** pretrained weights are NOT baked in (academic-data provenance uncleared); stage at runtime. The untracked `cloudbuild.cr.yaml` is a convenience build config hardcoding the owner's AR path/region.

`scripts/verify_compute_path.py` ? what it asserts

Submits `/api/process` with `--compute local|cloud`, polls `/api/jobs/{id}` to terminal, then asserts `result["compute"]["path"] == --expect` (`in_process|cloud_run`); exits 4 on mismatch (proves no silent fallback). With `--verify-gcp` it shells `gcloud run jobs executions describe <execution>` to confirm the recorded Cloud Run execution exists in GCP (exit 6 if not). Stdlib-only (urllib).

Files

- `C:/Users/avidu/Projects/bhi-computeval/backend/main.py` (`:273`, `:347-381`, `:489-503`, `:506-619`, `:703-708`, `:744-757`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/compute/__init__.py` (`:32-70`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/compute/cloud_run.py`
- `C:/Users/avidu/Projects/bhi-computeval/backend/config/presets.py` (`:51-56`), `backend/config/models.py` (`:311-368`), `backend/config/loader.py` (`:167-187`)
- `C:/Users/avidu/Projects/bhi-computeval/backend/worker/__main__.py` (`:118-170`, `:263-268`, `:496-501`)
- `C:/Users/avidu/Projects/bhi-computeval/deploy/cloudrun/README.md`, `deploy/cloudrun/Dockerfile`
- `C:/Users/avidu/Projects/bhi-computeval/scripts/verify_compute_path.py`
- `C:/Users/avidu/Projects/bhi-computeval/tests/test_api_cloud_dispatch.py` (23 pass)

NOT implemented / honest gaps

- `player_pool` and `max_frames` are **dropped on the cloud path** (not threaded to the worker `infer` CLI) ? `main.py:573` documents this as a follow-up.
- No real Cloud Run execution has been run (M7 owner step pending; `GoogleCloudRunJobClient` is `pragma: no cover`, no deploy runlog exists).
- The whole feature is on an **unmerged branch**; `origin/master` does not yet have the `compute` request field, `cloud_available`, or the verify script.
- WASB weights provenance (WASB-C3) is unresolved ? owner-gated before any public, non-owner cloud serving.